KNN & PCA

1.What is K-Nearest Neighbors (KNN) and how does it work in both classification and regression problems?
Ans: K-Nearest Neighbors (KNN) is a supervised learning algorithm used for both classification and regression. It works by finding the K closest data points (neighbors) to a new input based on a distance metric (like Euclidean distance).

- In classification: the output is determined by majority voting among the K nearest neighbors.
- In regression: the output is the average value of the K nearest neighbors.

KNN is simple, does not require training, and depends heavily on the choice of K and distance measure.

2.What is the Curse of Dimensionality and how does it affect KNN performance?
Ans: Curse of Dimensionality refers to the problem that arises when the number of features (dimensions) in a dataset increases, making the data sparse and distances between points less meaningful.

Effect on KNN:

- KNN relies on distance to find nearest neighbors
- In high dimensions, all points become almost equally distant
- This makes it hard to identify true nearest neighbors
- As a result:
    - Accuracy decreases
    - Computation increases
    - Model may perform poorly

3.: What is Principal Component Analysis (PCA)? How is it different from feature selection?
Ans: Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms the original features into a new set of uncorrelated variables called *principal components*. These components capture the maximum variance in the data using fewer dimensions.

Difference from Feature Selection:

- PCA (Feature Extraction):
    - Creates new features (principal components)
    - Combines original variables

    - May lose interpretability
- Feature Selection:
    - Selects important features from existing ones
    - Does not create new features
    - Keeps interpretability intact

4.What are eigenvalues and eigenvectors in PCA, and why are they important?

In Principal Component Analysis (PCA):

🔹 Eigenvectors

- They represent the directions (principal components) in which the data varies the most
- Each eigenvector defines a new axis in the transformed feature space

🔹 Eigenvalues

- They represent the amount of variance captured by each eigenvector
- Higher eigenvalue ⇒ more important principal component

🔹 Why they are important:

- PCA uses eigenvectors to find new feature directions
- Eigenvalues help rank these directions based on importance
- We select top components (with highest eigenvalues) to reduce dimensions while keeping maximum information

5.How do KNN and PCA complement each other when applied in a single pipeline?
Ans:
What is K-Nearest Neighbors (KNN) and how does it work in both classification and regression problems?

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for both classification and regression. It is a non-parametric and lazy learning method, meaning

it doesn’t build an explicit model during training—it simply stores the data and makes decisions at prediction time.

How KNN Works

1. Choose a value of K (number of neighbors)
2. Calculate the distance between the new data point and all training points (commonly using Euclidean distance)
3. Select the K nearest neighbors
4. Make prediction based on those neighbors

Distance Formula

$$d=\sum_{i=1}^{n}(x_i-y_i)^2$$

This is the Euclidean distance, used to measure how close two points are.

KNN for Classification

- Used when output is categorical (e.g., Yes/No, Spam/Not Spam)
- After finding K neighbors:

Example:

If K = 5 and among nearest neighbors:

- 3 are Class A
- 2 are Class B

Final prediction = Class A

KNN for Regression

- Used when output is continuous (e.g., price, salary)
- After finding K neighbors:

  Example:

If K = 3 and values are:

- 100, 120, 130

Prediction = (100 + 120 + 130) / 3 = 116.67

6.Dataset: Use the Wine Dataset from sklearn.datasets.load_wine(). Question 6: Train a KNN Classifier on the Wine dataset with and without feature scaling. Compare model accuracy in both cases. (Include your Python code and output in the code box below.)

Ans:

```python
# --------------------------
# 2. KNN WITH SCALING
# --------------------------
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

knn2 = KNeighborsClassifier(n_neighbors=5)
knn2.fit(X_train_scaled, y_train)
y_pred2 = knn2.predict(X_test_scaled)
acc2 = accuracy_score(y_test, y_pred2)

# --------------------------
# RESULTS
# --------------------------
print("Accuracy without scaling:", acc1)
print("Accuracy with scaling:", acc2)
```

```
Accuracy without scaling: 0.7222222222222222
Accuracy with scaling: 0.9444444444444444
```

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# Load dataset
wine = load_wine()
X = wine.data
y = wine.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# --------------------------
# 1. KNN WITHOUT SCALING
# --------------------------
knn1 = KNeighborsClassifier(n_neighbors=5)
knn1.fit(X_train, y_train)
y_pred1 = knn1.predict(X_test)
acc1 = accuracy_score(y_test, y_pred1)

# --------------------------
# 2. KNN WITH SCALING
# --------------------------
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

knn2 = KNeighborsClassifier(n_neighbors=5)
knn2.fit(X_train_scaled, y_train)
y_pred2 = knn2.predict(X_test_scaled)
```

Conclusion

KNN performs significantly better after feature scaling because it is a distance-based algorithm. Without scaling, features with larger ranges dominate distance calculations, leading to lower accuracy. Scaling ensures all features contribute equally, improving model performance.

7.Train a PCA model on the Wine dataset and print the explained variance ratio of each principal component. (Include your Python code and output in the code box below.)

Ans:

```python
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Load dataset
wine = load_wine()
X = wine.data

# Standardize the features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Apply PCA
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

# Explained variance ratio
print("Explained Variance Ratio of Each Principal Component:")
for i, ratio in enumerate(pca.explained_variance_ratio_, start=1):
    print(f"PC{i}: {ratio:.4f}")
```

```
Explained Variance Ratio of Each Principal Component:
PC1: 0.3620
PC2: 0.1921
PC3: 0.1112
PC4: 0.0707
PC5: 0.0656
PC6: 0.0494
PC7: 0.0424
PC8: 0.0268
PC9: 0.0222
PC10: 0.0193
PC11: 0.0174
PC12: 0.0130
PC13: 0.0080
```

8.Train a KNN Classifier on the PCA-transformed dataset (retain top 2 components). Compare the accuracy with the original dataset. (Include your Python code and output in the code box below.)

Ans:

```python
y_pred = knn.predict(X_test_scaled)
acc_original = accuracy_score(y_test, y_pred)

# ----------------------------
# 2. PCA (2 components) + KNN
# ----------------------------
pca = PCA(n_components=2)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

knn_pca = KNeighborsClassifier(n_neighbors=5)
knn_pca.fit(X_train_pca, y_train)

y_pred_pca = knn_pca.predict(X_test_pca)
acc_pca = accuracy_score(y_test, y_pred_pca)

# ----------------------------
# Results
# ----------------------------
print("Accuracy Comparison:")
print(f"Original Dataset Accuracy: {acc_original:.4f}")
print(f"PCA (2 Components) Accuracy: {acc_pca:.4f}")
```

```
Accuracy Comparison:
Original Dataset Accuracy: 0.9444
PCA (2 Components) Accuracy: 1.0000
```

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load dataset
wine = load_wine()
X = wine.data
y = wine.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----------------------------
# 1. KNN on ORIGINAL dataset
# ----------------------------
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)

y_pred = knn.predict(X_test_scaled)
acc_original = accuracy_score(y_test, y_pred)

# ----------------------------
# 2. PCA (2 components) + KNN
# ----------------------------
pca = PCA(n_components=2)
```

9. Train a KNN Classifier with different distance metrics (euclidean, manhattan) on the scaled Wine dataset and compare the results. (Include your Python code and output in the code box below.)

Ans:

```python
# ----------------------------
# KNN with Euclidean distance
# ----------------------------
knn_euclidean = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
knn_euclidean.fit(X_train_scaled, y_train)
y_pred_euclidean = knn_euclidean.predict(X_test_scaled)
acc_euclidean = accuracy_score(y_test, y_pred_euclidean)

# ----------------------------
# KNN with Manhattan distance
# ----------------------------
knn_manhattan = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=1)
knn_manhattan.fit(X_train_scaled, y_train)
y_pred_manhattan = knn_manhattan.predict(X_test_scaled)
acc_manhattan = accuracy_score(y_test, y_pred_manhattan)

# ----------------------------
# Results
# ----------------------------
print("KNN Distance Metric Comparison:")
print(f"Euclidean (p=2) Accuracy : {acc_euclidean:.4f}")
print(f"Manhattan (p=1) Accuracy: {acc_manhattan:.4f}")
```

```
KNN Distance Metric Comparison:
Euclidean (p=2) Accuracy : 0.9444
Manhattan (p=1) Accuracy: 0.9444
```

```python
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Load dataset
wine = load_wine()
X = wine.data
y = wine.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature Scaling (important for KNN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# ----------------------------
# KNN with Euclidean distance
# ----------------------------
knn_euclidean = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2)
knn_euclidean.fit(X_train_scaled, y_train)
y_pred_euclidean = knn_euclidean.predict(X_test_scaled)
acc_euclidean = accuracy_score(y_test, y_pred_euclidean)

# ----------------------------
# KNN with Manhattan distance
# ----------------------------
knn_manhattan = KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=1)
```

10.You are working with a high-dimensional gene expression dataset to classify patients with different types of cancer. Due to the large number of features and a small number of samples, traditional models overfit. Explain how you would: ● Use PCA to reduce dimensionality ● Decide how many components to keep ● Use KNN for classification

post-dimensionality reduction ● Evaluate the model ● Justify this pipeline to your stakeholders as a robust solution for real-world biomedical data (Include your Python code and output in the code box below.)

Ans: 1. Using PCA for Dimensionality Reduction

Gene expression datasets contain a very large number of features (genes) but a small number of samples. This leads to:

- Overfitting
- High computational cost
- Noisy and redundant features

To handle this, Principal Component Analysis (PCA) is used:

- Standardize the dataset first
- Transform original features into principal components
- Retain only components that capture most of the variance in the data

This reduces dimensionality while preserving important patterns in the dataset.

---

2. Deciding Number of Components

The number of principal components is selected based on:

- Cumulative explained variance ratio

A common approach is:

- Retain components that explain 90%–95% of total variance

This ensures:

- Minimal information loss
- Maximum reduction in complexity

---

3. Using KNN After PCA

After dimensionality reduction:

- K-Nearest Neighbors (KNN) is applied for classification

Reasons:

- Works well in lower-dimensional spaces
- Sensitive to distance → performs better after noise removal
- Simple and effective for biomedical classification tasks

Best practices:

- Use scaled and PCA-transformed data
- Choose optimal value of K (commonly 3, 5, 7)

---

4. Model Evaluation

The model is evaluated using:

- Accuracy score
- Confusion matrix
- Cross-validation accuracy

These metrics ensure:

- Model generalization
- Balanced performance across classes
- Robustness on unseen data

---

5. Justification of Pipeline (PCA + KNN)

This pipeline is suitable for biomedical datasets because:

- Reduces high dimensional gene data into meaningful components
- Prevents overfitting due to small sample size
- Improves computational efficiency
- Preserves important variance (biological signal)
- Enhances KNN performance by reducing noise

Thus, PCA + KNN provides a simple, interpretable, and effective solution for cancer classification problems in high-dimensional biomedical data.

```python
knn.fit(X_train_pca, y_train)

# Predictions
y_pred = knn.predict(X_test_pca)

# -------------------------
# Evaluation
# -------------------------
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

# Cross-validation
cv_scores = cross_val_score(knn, X_train_pca, y_train, cv=5)

print("\nModel Performance:")
print(f"Test Accuracy: {acc:.4f}")
print("Confusion Matrix:\n", cm)
print(f"Cross-validation Accuracy: {np.mean(cv_scores):.4f}")
```

```
Number of components selected: 10

Model Performance:
Test Accuracy: 0.9561
Confusion Matrix:
 [[40  3]
 [ 2 69]]
Cross-validation Accuracy: 0.9538
```

```python
# Standardization
# -------------------------
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# -------------------------
# PCA (choose components for 95% variance)
# -------------------------
pca = PCA(n_components=0.95)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Number of components selected:", pca.n_components_)

# -------------------------
# KNN model
# -------------------------
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_pca, y_train)

# Predictions
y_pred = knn.predict(X_test_pca)

# -------------------------
# Evaluation
# -------------------------
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

# Cross-validation
cv_scores = cross_val_score(knn, X_train_pca, y_train, cv=5)

print("\nModel Performance:")
print(f"Test Accuracy: {acc:.4f}")
print("Confusion Matrix:\n", cm)
```

```python
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import numpy as np

# -------------------------
# Load dataset
# -------------------------
data = load_breast_cancer()
X = data.data
y = data.target

# -------------------------
# Train-test split
# -------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -------------------------
# Standardization
# -------------------------
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# -------------------------
# PCA (choose components for 95% variance)
# -------------------------
pca = PCA(n_components=0.95)
```